

Assignment Summary

Week 7 - Delta Lake, Customer Incremental Processing, and SCD Implementation

Project Area	Details
Main Focus	Delta Lake processing with customer master and incremental customer updates
Dataset Used	Superstore-style sales data: 9994 rows, 21 columns, 793 unique customers, and 5009 orders
Business Entity	Customer dimension built using customer ID, customer name, segment, country, city, state, postal code, and region
Tech Used	Python, PySpark, Delta Lake, pandas fallback, and Jupyter Notebook
Core Outputs	Clean customer data, SCD Type 1 table, SCD Type 2 history table, and validation outputs

1. Objective

The goal of this assignment was to understand how Delta Lake can be used for reliable incremental data processing. The work focuses on taking an existing customer master dataset, applying a new incremental customer file, and then handling the changes using both SCD Type 1 and SCD Type 2 approaches.

In simple terms, SCD Type 1 keeps only the latest customer details, while SCD Type 2 keeps the old customer record as history and inserts a new active version whenever tracked fields change.

2. Dataset Preparation

The original uploaded file contains order-level Superstore data. Since the assignment is focused on customer-level SCD processing, the notebook first extracts customer-related fields from the sales dataset and converts them into a customer dimension structure.

- A customer master file is prepared from the initial customer records.
- A separate incremental file is created to simulate new customers and changed customer details.
- Only customer-level attributes are used for SCD tracking, so product and order-level columns are not directly used in the merge logic.
- Basic cleaning is applied to handle blank values, duplicate records, and inconsistent text formatting before loading the data.

3. Data Loading and Cleaning Flow

The notebook reads the customer master and customer incremental CSV files from the data folder. After loading, it standardizes column names, trims text fields, fills missing values where required, and removes duplicate customer records. This keeps the SCD logic simple and prevents duplicate or dirty records from creating incorrect history.

The cleaned data is then written into Delta format when Spark and Delta are available. A pandas fallback is also included so the assignment can still run locally if the system has Java, Spark, or Delta dependency issues.

4. SCD Type 1 Implementation

SCD Type 1 is used when the latest value is more important than maintaining older versions. In this assignment, the current customer record is updated directly when a matching customer ID is found in the incremental file. If the customer does not already exist, the record is inserted as a new customer.

- Existing customer records are updated with the latest segment, location, and region information.
- New customer IDs from the incremental file are inserted into the final customer table.
- No historical version is maintained in SCD Type 1, so it represents the most current customer view.

5. SCD Type 2 Implementation

SCD Type 2 is used when it is important to preserve the earlier customer information. For this, the notebook creates additional columns such as effective start date, effective end date, is current, and record hash. The hash helps identify whether a tracked customer attribute has actually changed.

- If a customer is new, a new active record is inserted.
- If a customer already exists and the tracked attributes have changed, the old active record is expired.
- A new active version is inserted with the updated customer details.
- The old version remains available for historical tracking and audit purposes.

6. Validation Performed

After the SCD operations, validation steps are included to check whether the final output makes sense. The notebook checks row counts, current and expired records, changed customers, and final table previews. These checks help confirm that the merge logic is working instead of simply assuming the output is correct.

7. Final Output

The assignment produces customer-level output tables for both SCD Type 1 and SCD Type 2. The SCD Type 1 output gives the latest customer snapshot, while the SCD Type 2 output gives both current and historical customer versions. This structure is useful in real data engineering work because business teams often need both the current state and past changes for reporting.

8. Key Learning

This assignment helped connect Delta Lake concepts with a practical customer data use case. The main learning was not just how to write merge code, but why incremental processing and SCD handling matter in real pipelines. Customer details can change over time, and the data model should clearly decide whether to overwrite the old value or preserve it as history.

Overall, the project follows a clean flow: read data, clean data, load to Delta, apply SCD Type 1, apply SCD Type 2, validate the result, and save the final outputs.